

과제 1 — 정렬 비교: 병합 · 퀵 · 카테일 셰이커

2026-2 고급알고리즘(SIT2001-01) · 정시후 · 2026-09-30

- **GitHub 저장소:** <https://github.com/ericsihu-ship-it/algorithms-hw1>
- 비교한 정렬: 병합 정렬 · 퀵 정렬(배운 정렬) + **카테일 셰이커 정렬(배우지 않은 정렬)**
- 실행: `make test` (유닛 테스트 89개) · `make run` (실험 4개) · `make charts` (그래프)

요약

	병합 정렬	퀵 정렬	카테일 셰이커 정렬
구분	배운 정렬 (주제 03)	배운 정렬 (주제 04)	배우지 않은 정렬
평균 · 최악	$O(n \log n) \cdot O(n \log n)$	$O(n \log n) \cdot O(n^2)$	$O(n^2) \cdot O(n^2)$
최선	$O(n \log n)$	$O(n \log n)$	$O(n)$ (정렬된 입력)
추가 공간	$O(n)$ (temp 배열)	$O(\log n)$ (재귀 스택)	$O(1)$
안정성 (실측)	안정	불안정	안정
실험에서 드러난 약점	메모리를 n 칸 더 쓴다	같은 값이 많으면 $O(n^2)$	무작위 입력에서 병합보다 100배 넘게 느리다

1. **$O(n^2)$ 의 벽.** n 을 두 배로 늘리면 카테일 정렬의 비교는 약 4배, 병합 · 퀵은 약 2.2배로 늘었다. $n = 32,768$ 에서 카테일 정렬은 병합 정렬보다 비교를 약 900배 더 했다.
2. **카테일 정렬이 버블 정렬에서 고친 것은 "거북이" 하나다.** 맨 끝에 최솟값이 있는 입력에서 비교가 3,199만 번에서 2만 4천 번으로 줄었다. 교환 수는 어떤 입력에서도 줄지 않았다.
3. **퀵 정렬은 입력을 탄다.** 수업 코드의 파티션은 같은 값이 많으면 피벗을 어떻게 골라도 $O(n^2)$ 이 됐다. 네 입력 모두에서 버틴 조합은 랜덤 피벗 + 3-way 파티션 하나뿐이었다.

1. 구현과 검증

1.1 공통 인터페이스

세 정렬을 같은 조건으로 재려고 모두 `qsort`와 같은 모양의 함수로 만들고, 함수 포인터를 담은 구조체 표 하나로 부른다. 실험 프로그램과 테스트는 정렬 이름을 적지 않고 이 표만 훑는다. 이 구조는 샘플 저장소(hw1-sample-2026)를 참고했다.

```
typedef struct SortAlgorithm {
    const char *name;
    const char *averageTime, *worstTime, *extraSpace;
    int stable; /* 안정 정렬이라고 "주장"하는 값. 테스트가 실측과 맞춰 본다 */
    void (*sort)(void *base, size_t n, size_t size, SortCompare cmp, SortStats *stats);
} SortAlgorithm;
```

- **세는 곳이 한 곳이다.** 정렬 코드는 원소를 `sortCompare · sortCopy · sortSwap(src/sortutil.h)`으로만 만진다. 그래서 세 정렬의 비교 · 이동을 같은 규칙으로 센다. 교환 한 번은 이동 3회다(주제 02).
- **안정성을 실측한다.** 원소를 (key, order)로 만들어 key로만 정렬하고, 같은 key끼리 입력 순서(order)가 남았는지 본다.
- **입력과 피벗이 재현된다.** 난수는 주제 04의 `minstd(x ← 16807x mod (231-1))`를 직접 구현해 쓴다. 비교 · 이동 횟수는 어느 기계에서 돌려도 같다. 기계를 타는 것은 시간뿐이다.
- 병합 · 퀵은 수업 코드(algorithm-code)의 절차를 원소 타입과 무관하게 옮겼다. 각 정렬의 의사코드는 `src/*.pseudo`에 있다.

1.2 검증

유닛 테스트 89개가 세 정렬, 퀵 변형 여섯, 기준선 버블 정렬 모두에 같은 검사를 돌린다. 기본 케이스, `qsort` 대조, $n = 0..300$ 전수, 여섯 입력 모양, 안정성 주장과 실측의 일치를 본다. 그중 가장 중요한 것은 **수업의 숫자가 그대로 나오는가**다. 원소 타입과 무관하게 옮긴 코드가 수업 코드와 같은 규칙으로 센다는 증거가 된다.

정렬	입력 (n = 10)	수업 숫자 (비교 · 이동)	이 구현
병합	예제 · 정렬됨 · 역순	22·68 / 19·68 / 15·68	같음
퀵 (첫 원소 피벗)	최선 · 예제 · 정렬됨 · 역순	19·9 / 25·21 / 45·0 / 45·15	같음
버블 (+ flag)	정렬됨 / 주제 02 · 03 예제	비교 9 / 이동 87 · 75	같음

테스트가 실제로 버그를 잡는지도 확인했다. 코드를 일부러 망가뜨리자 모두 실패로 드러났다.

망가뜨린 곳	결과
병합의 <= 를 < 로	안정성 검사 실패 (정렬 결과는 멀쩡하다)
카테일 정렬의 > 를 >= 로	안정성 검사 등 2개 실패
카테일 정렬의 뒤로 가는 회전을 지우기	기본 케이스 등 8개 실패
교환에서 i == j 를 건너뛰지 않게	퀵 정렬의 수업 이동 횟수 4개 불일치

ASan · UBSan을 붙인 테스트(make sanitize)도 경고 없이 통과했다.

2. 카테일 셰이커 정렬 — AI로 배운 것

배우지 않은 정렬은 Wikipedia의 정렬 비교표에서 카테일 셰이커 정렬(cocktail shaker sort)을 골랐다. 버블 정렬의 변형이라, 버블 정렬과 무엇이 다르고 그 차이가 실제로 얼마나 되는지가 핵심 질문이었다. 아래 표는 AI(Claude)가 설명한 내용을 질문별로 요약하고, 각 설명을 이 저장소의 실험으로 확인한 결과를 붙인 것이다.

질문	AI 설명 (요약)	실험으로 확인한 것
어떤 정렬인가?	버블 정렬을 앞 · 뒤로 번갈아 돈다. 앞으로 가며 가장 큰 값을 뒤로, 뒤로 가며 가장 작은 값을 앞으로 보낸다. 카테일 셰이커를 흔드는 모양이라 이 이름이 붙었다 (양방향 버블 정렬, shaker sort).	구현 · 의사코드 (아래)
버블 정렬의 무엇을 고쳤나?	거북이 문제. 버블 정렬에서 뒤쪽의 작은 값은 한 회전에 한 칸씩만 앞으로 온다. 앞쪽의 큰 값(토끼)은 한 회전이면 끝까지 간다. 카테일 정렬은 뒤로 가는 회전 한 번에 거북이를 맨 앞까지 옮긴다.	실험 4: 거북이 입력에서 비교 31,996,000 → 23,994 (n = 8,000). 토끼 입력에서는 두 정렬이 같다
시간복잡도는?	교환이 없으면 멈추므로 최선(정렬됨)은 O(n). 평균 · 최악은 버블 정렬과 같은 O(n²)이다. 차수는 바뀌지 않는다.	실험 1: 정렬됨 입력 비교 9,999 = n-1. 실험 2: n이 두 배일 때 비교 약 4배
교환도 줄어드나?	아니다. 이웃끼리 한 번 바꿀 때마다 역순 쌍(inversion)이 정확히 하나 줄어드므로, 교환 수는 역순 쌍 수와 같다. 버블 정렬과 똑같다.	실험 4: 다섯 입력 모두에서 이동이 버블 정렬과 한 자리까지 같다 (무작위 48,239,817)
버블보다 얼마나 빠른가?	비교와 회전이 줄어드는 만큼 조금 빠르지만 차수는 같다.	무작위 n = 8,000: 비교 24% 적고 시간 22% 짧다 (208.7 → 163.4 ms). 역순은 똑같다
안정성 · 공간은?	이웃끼리 > 일 때만 바꾸므로 안정, 교환용 한 칸만 쓰므로 O(1).	실험 1: 중복많은 입력에서 안정, 추가 메모리 8 B
실제로 쓰이나?	거의 안 쓰인다. 교육용이고, 작은 배열에는 보통 삽입 정렬이 쓰인다.	확인하지 않았다

```

ALGORITHM CocktailShakerSort(A, n)           // src/cocktailShakerSort.pseudo
    lo <- 0, hi <- n-1, swapped <- true
    while swapped do
        swapped <- false
        for i <- lo to hi-1 do                 // 앞으로: 가장 큰 값이 A[hi]로
            if A[i] > A[i+1] then swap A[i], A[i+1]; swapped <- true
        if not swapped then break             // 교환이 없으면 이미 정렬됐다
        hi <- hi - 1
        swapped <- false
        for i <- hi-1 downto lo do             // 뒤로: 가장 작은 값이 A[lo]로
            if A[i] > A[i+1] then swap A[i], A[i+1]; swapped <- true
        lo <- lo + 1

```

버블 정렬과의 차이를 공정하게 보려고, 수업의 버블 정렬에 주제 02 "개선 연습"의 flag를 넣은 판을 **기준선**으로 따로 두었다 (src/bubbleSort.c). 그러면 두 정렬의 차이는 "뒤로 가는 회전" 하나만 남는다. 기준선은 실험 4에서만 쓴다.

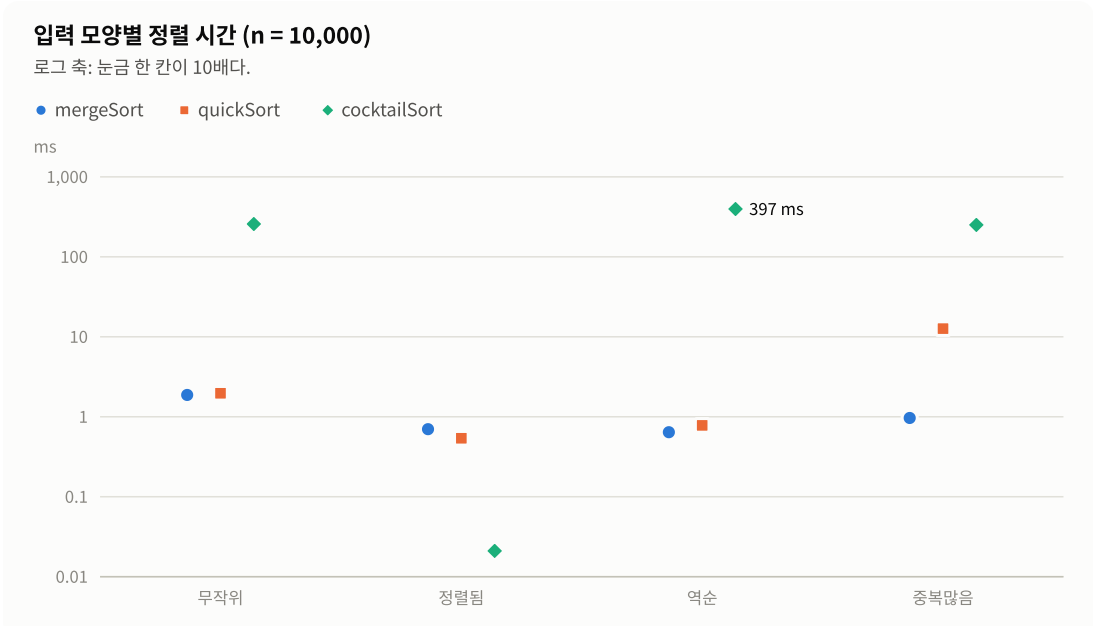
3. 실험 결과

측정 환경은 클라우드 리눅스 컨테이너(Intel Xeon 2.1 GHz, 2코어), gcc 13.3, -O2 다. 원소는 (key, order) 8바이트다. 시간은 clock() 으로 정렬 구간만 여러 번 재어 가운데 값을 썼다. 입력은 무작위(minstd 난수, 키가 사실상 겹치지 않는다) · 정렬됨 · 역순 · 중복많음(키가 0~7의 8가지)과, 실험 4의 거북이 · 토끼다. 시드가 고정되어 있어 입력은 매번 같다. 측정값 원본은 저장소의 report/data/*.csv 이고, 그래프는 make charts 로 다시 만든다.

3.1 실험 1 — 입력 모양별 (n = 10,000)

입력	정렬	시간(ms)	비교	이동	추가 메모리	재귀 깊이	안정
무작위	병합	1.88	120,461	267,232	80,000 B	14	-
	퀵	1.97	162,030	277,362	8 B	31	-
	칵테일	257.30	37,696,680	75,214,368	8 B	1	-
정렬됨	병합	0.70	69,008	267,232	80,000 B	14	-
	퀵	0.54	155,798	29,994	8 B	29	-
	칵테일	0.02	9,999	0	8 B	1	-
역순	병합	0.64	64,608	267,232	80,000 B	14	-
	퀵	0.78	159,885	168,780	8 B	33	-
	칵테일	396.67	49,995,000	149,985,000	8 B	1	-
중복많음	병합	0.97	115,560	267,232	80,000 B	14	○
	퀵	12.68	6,285,530	74,451	8 B	1,295	×
	칵테일	250.94	37,602,269	65,411,724	8 B	1	○

안정 열(○ 안정, × 불안정)은 같은 키가 많은 중복많음에서만 의미가 있어, 나머지는 "-"로 적었다. 키가 겹치지 않으면 불안정 정렬도 순서를 지킨 것처럼 보이기 때문이다. 추가 메모리는 정렬이 따로 잡은 메모리(병합의 temp 배열, 교환용 한 칸)이고, 퀵 정렬의 재귀 스택은 재귀 깊이 열로 본다.



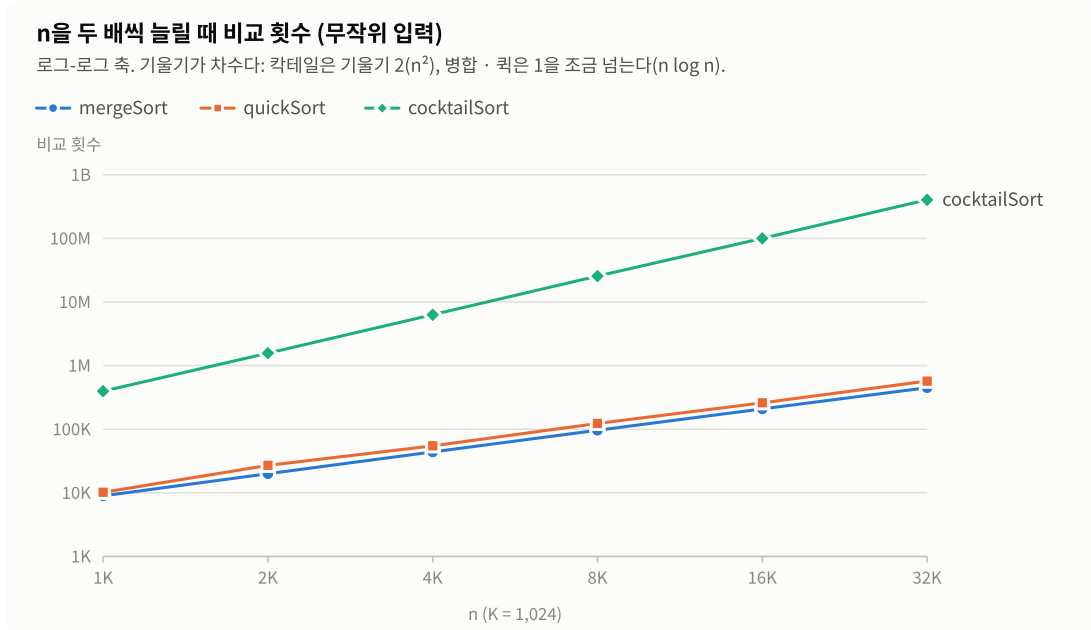
- **칵테일 정렬은 양 끝을 오간다.** 정렬된 입력에서는 비교 n-1번(9,999)으로 끝나 셋 중 가장 빠르다(0.02 ms). 무작위에서는 병합보다 137배, 역순에서는 618배 느리다.
- **병합 정렬의 이동은 입력과 무관하게 267,232회로 같다.** 병합마다 구간 전체가 temp 배열로 갔다가 돌아오기 때문이다(주제 03). 대신 temp로 n칸(80,000 B)을 더 쓴다.

- 퀵 정렬은 중복많음에서 무너진다. 비교가 무작위일 때의 39배(6,285,530)이고 재귀 깊이가 1,295다. 원인은 실험 3에서 찾았다.

3.2 실험 2 — 배가 실험 (무작위 입력, n을 두 배씩)

n	병합 비교 (배율)	퀵 비교 (배율)	카테일 비교 (배율)	카테일 시간
1,024	8,944	10,178	396,011	2.3 ms
2,048	19,954 (×2.23)	26,990 (×2.65)	1,567,222 (×3.96)	10.9 ms
4,096	43,945 (×2.20)	54,811 (×2.03)	6,300,657 (×4.02)	45.0 ms
8,192	96,127 (×2.19)	123,358 (×2.25)	25,468,126 (×4.04)	175.6 ms
16,384	208,721 (×2.17)	260,057 (×2.11)	99,934,476 (×3.92)	726.4 ms
32,768	450,202 (×2.16)	571,495 (×2.20)	404,115,457 (×4.04)	2,988.7 ms

배율은 바로 위 행(n이 절반)과 견준 값이다. $n \log n$ 이면 2를 조금 넘고, n^2 이면 4다(배가 실험).

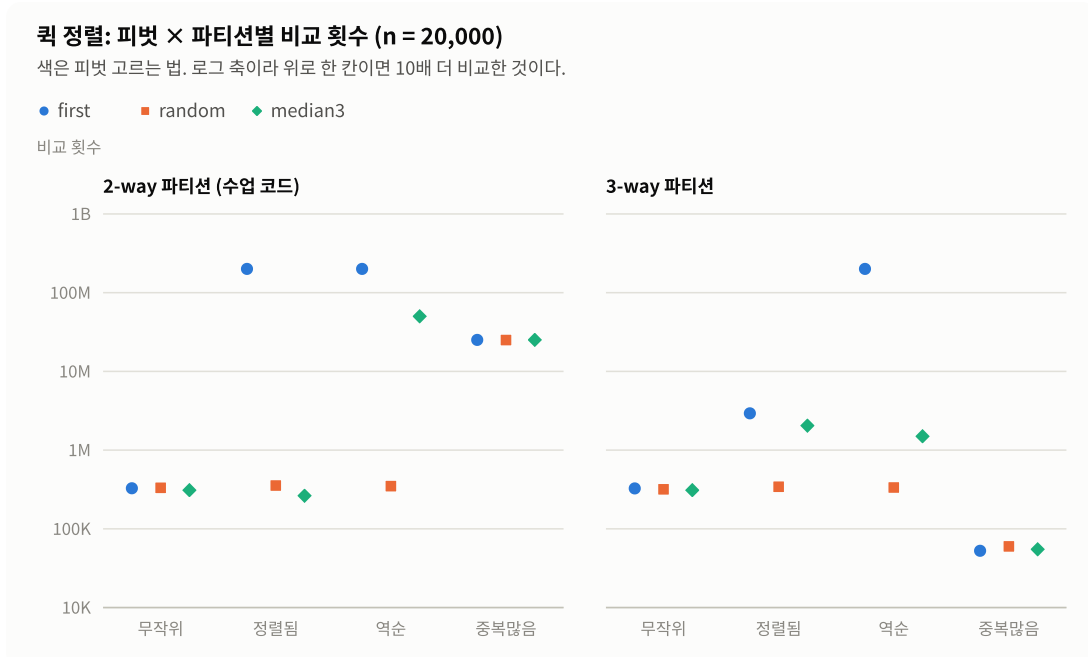


- 카테일 정렬의 비교는 n이 두 배가 될 때마다 **3.92~4.04배**로 늘었다. $O(n^2)$ 이다. 병합은 **2.16~2.23배**, 퀵은 **2.03~2.65배**로 $n \log n$ 의 모양이다.
- 로그-로그 축에서 카테일 정렬의 기울기는 2, 병합·퀵은 1을 조금 넘는다. $n = 32,768$ 에서 카테일 정렬은 병합 정렬보다 비교를 898배, 시간을 564배 더 썼다(2,988.7 ms 대 5.3 ms). 버블 정렬을 아무리 고쳐도 넘지 못하는 벽을, 분할 정복은 나누는 것만으로 넘는다(주제 03).

3.3 실험 3 — 퀵 정렬: 피벗 × 파티션 (n = 20,000, 비교 횟수)

피벗 · 파티션	무작위	정렬됨	역순	중복많음
첫 원소 · 2-way (수업 코드)	327,778	199,990,000	199,990,000	25,067,417
랜덤 · 2-way (본 비교의 퀵 정렬)	332,305	354,792	349,347	25,054,708
셋 중 가운데 · 2-way	309,740	263,630	50,034,998	25,159,941
첫 원소 · 3-way	326,101	2,932,763	199,990,000	52,506
랜덤 · 3-way	317,859	342,280	335,585	59,786
셋 중 가운데 · 3-way	309,651	2,047,602	1,499,334	54,979

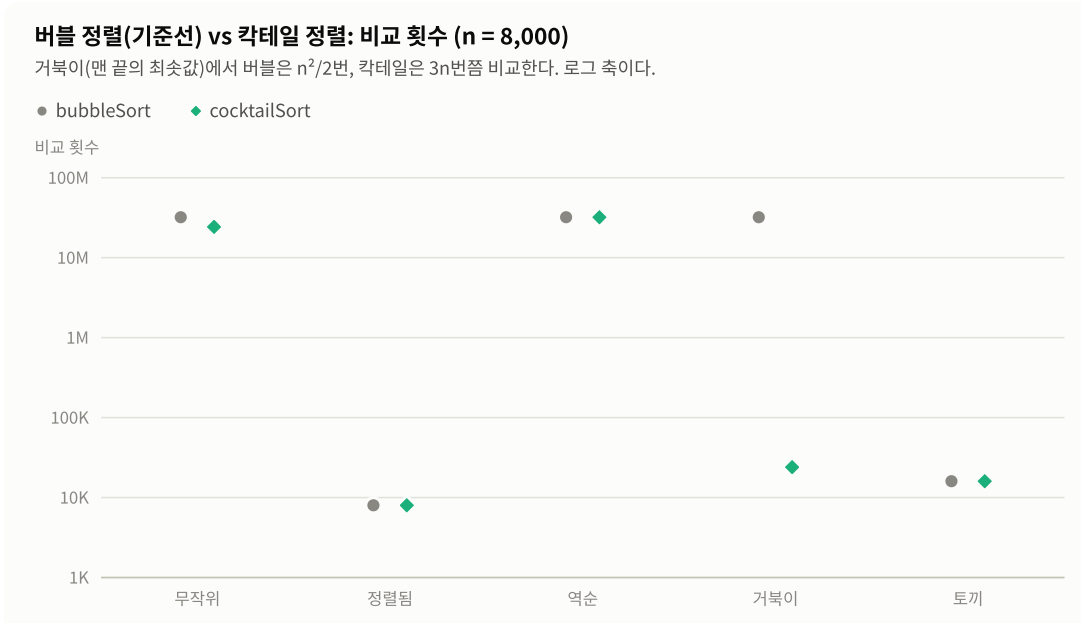
굵은 칸은 $O(n^2)$ 로 무너진 칸이다(n이 두 배면 비교가 4배). 기울인 칸은 n이 두 배일 때 비교가 약 2.8배씩 늘어 $n \log n$ (약 2.1배)을 벗어난 칸이다.



- **정렬된 입력 + 첫 원소 피벗**은 수업대로 $n(n-1)/2 = 199,990,000$ 번 비교했다. 랜덤 피벗이 이것을 고친다(주제 04).
- **중복많은**은 피벗으로 못 고친다. 수업의 파티션은 피벗보다 작은 것만 왼쪽으로 보내고 같은 값은 오른쪽에 남긴다. 키가 8가지뿐이라 같은 값끼리 모인 구간이 생기고, 그 구간에서는 파티션마다 피벗 하나만 떨어져 나간다. 어떤 원소를 골라도 같은 값이라 피벗 고르는 법과 무관하다. 3-way 파티션(작은 것 | 같은 것 | 큰 것)은 같은 값을 한 번에 제자리에 놓아 비교를 $25,054,708 \rightarrow 59,786$ 으로 줄였다(랜덤 피벗).
- **셋 중 가운데도 만능은 아니다.** 역순 입력을 수업의 파티션으로 한 번 나누면 왼쪽 구간이 [최솟값, 나머지는 내림차순] 모양이 된다. 이 구간에서 셋 중 가운데는 두 번째로 작은 값이 되고, 다음 구간에서는 두 번째로 큰 값이 된다. 파티션마다 한두 원소만 떨어져 나가 비교 50,034,998번, 재귀 깊이 5,001이 되었다($n = 16$ 에서 단계별로 따라가 확인).
- 네 입력 모두에서 $n \log n$ 을 지킨 것은 **랜덤 · 3-way** 하나뿐이었다. 본 비교의 퀵 정렬은 수업 코드에 가깝게 랜덤 · 2-way로 두었고, 실험 1의 중복많은에서 무너진 이유가 이것이다.

3.4 실험 4 — 버블 정렬(기준선) vs 카테일 정렬 (n = 8,000)

입력	버블 비교	카테일 비교	줄어든 비율	이동 (둘이 같다)
무작위	31,994,674	24,259,789	24%	48,239,817
정렬됨	7,999	7,999	0%	0
역순	31,996,000	31,996,000	0%	95,988,000
거북이 (1 2 ... n-1 뒤에 0)	31,996,000	23,994	99.9%	23,997
토끼 (n 뒤에 1 2 ... n-1)	15,997	15,997	0%	23,997



- **거북이에서만 차수가 바뀐다.** 버블 정렬은 맨 끝의 0을 한 회전에 한 칸씩 옮겨 $n(n-1)/2$ 번 비교했다. 칵테일 정렬은 앞 · 뒤 · 앞 세 회전, $3n-6 = 23,994$ 번에 끝냈다. 두 공식 모두 테스트가 $n = 10$ 에서 $45 \cdot 24$ 로 확인한다.
- **토끼는 둘 다 빠르다.** 맨 앞의 큰 값은 버블 정렬의 첫 회전에 맨 끝까지 간다. 그래서 칵테일 정렬이 더할 것이 없다.
- **이동은 다섯 입력 모두에서 같았다.** 이웃 교환 한 번이 역순 쌍을 정확히 하나 없애므로, 교환 수는 입력이 정한다. 칵테일 정렬이 줄이는 것은 회전 수, 곧 비교뿐이다. 무작위 입력에서 비교는 24%, 시간은 22% 줄었지만($208.7 \rightarrow 163.4$ ms) 이동 4,824만 번은 그대로라 $O(n^2)$ 을 벗어나지 못한다.

4. 결론

상황	고를 정렬	근거 (이 실험)
같은 키의 순서를 지켜야 한다	병합	셋 중 $O(n \log n)$ 이면서 안정한 것은 병합뿐이다. 입력과 무관하게 일정하다
메모리를 아끼고 평균이 빨라야 한다	퀵 (랜덤 피벗, 중복이 많으면 3-way)	추가 메모리 8 B. 수업의 파티션은 중복에서 $O(n^2)$ 이다
거의 정렬된 작은 배열, 설명용	칵테일 셰이커	정렬됨에서 비교 $n-1$ 번, 거북이를 한 번에 옮긴다. 그 밖에서는 $O(n^2)$ 이다

세 정렬은 저마다 무엇을 내주고 무엇을 얻는다. 병합은 메모리를 내주고 안정성과 일정한 성능을 얻었다. 퀵은 최악의 위험을 내주고 적은 메모리와 빠른 평균을 얻었다. 칵테일 셰이커 정렬은 버블 정렬의 거북이 문제를 고쳤지만, 교환 수와 $O(n^2)$ 의 차수는 그대로다. 차수의 차이는 회전 방향을 바꾸는 식의 개선으로는 메울 수 없었다. $n = 32,768$ 에서 비교 900배라는 차이가 그 증거다.

5. 한계

- 시간은 클라우드 컨테이너 한 대에서 켜 값이다. 다른 기계 · 최적화 수준에서는 달라진다. 그래서 비교 · 이동 횟수를 함께 실었다. 이 둘은 어디서 돌려도 같다.
- 칵테일 정렬은 교환이 없으면 멈추는 판이다. 마지막 교환 위치까지 구간을 줄이는 개선판은 구현하지 않았다.
- 퀵 정렬의 스택 사용량은 재귀 깊이로만 보았고, 바이트로 재지 않았다.
- 거북이 · 토끼 입력은 차이를 보이려고 만든 극단적인 입력이다. 실제 데이터에서 칵테일 정렬의 이득은 무작위 입력의 24% (비교) 쪽에 가깝다.

6. AI 활용

칵테일 셰이커 정렬의 학습(2장)과 코드 · 실험 · 보고서 초안 작성에 AI(Anthropic의 Claude)를 사용했다. 저장소의 커밋에도 공동 작성자로 표시되어 있다. 보고서의 모든 수치는 저장소 코드로 다시 쉼 수 있고, 구현은 유닛 테스트 89개와 수업 숫자 재현으로 검증했다.